

Bases de Datos 2 - TPI Parte 2 - Grupo 11

Matias Carro y Hugo Catalan

Documentación del Proyecto - Sistema de Gestión de Usuarios
(Node.js, Mongoose y MongoDB)

Integrantes: Hugo Catalan y Matias Carro

Materia: Bases de Datos 2

Comision 11

Grupo: TPI-G011

Profesor: Bruselario, Sebastián

Tutora:Maragliano, Julieta

Conexion al sistema y Base de datos.....	4
Desde mongosh:.....	4
Desde MongoDB Compass:.....	4
Credenciales de Administrador para el script con el CRUD de la base de datos:.....	5
Introducción del sistema.....	6
Tecnologías utilizadas.....	6
Arquitectura del proyecto.....	6
Descripción de archivos.....	7
• index.js.....	7
• MenuPrincipal.js.....	7
• TPI_parte2_mongoose.js.....	7
Entidades del sistema.....	7
Modelo de datos.....	8
Usuarios.....	8
Schema Usuarios.....	8
Descripción de campos.....	8
Profesionales.....	9
Schema Profesionales.....	9
Descripción de campos.....	9
Servicios.....	10
Schema Servicios.....	10
Descripción de campos.....	10
Turnos.....	11
Schema Turnos.....	11
Descripción de campos.....	11
Funciones CRUD.....	12
Usuarios.....	12
Crear usuario (CREATE).....	12
Listar usuarios activos (READ).....	13
Actualizar un usuario (UPDATE).....	13
Dar de baja un usuario (DELETE) Baja logica.....	13
Profesionales.....	13
Crear profesional (CREATE).....	13
Listar profesionales activos (READ).....	14
Actualizar un profesional (UPDATE).....	14
Dar de baja un profesional (DELETE) Baja lógica.....	14
Servicios.....	14
Crear servicio (CREATE).....	14
Listar servicios activos (READ).....	14
Listar servicios por profesional (READ).....	15
Actualizar un servicio (UPDATE).....	15
Dar de baja un servicio (DELETE) Baja lógica.....	15
Turnos.....	15
Crear turno (CREATE).....	15

Listar turnos activos (READ).....	17
Actualizar un turno (UPDATE).....	17
Dar de baja un turno (DELETE) Baja lógica.....	17
Guía de Uso del Sistema.....	18
Requisitos previos.....	18
Instalación de dependencias.....	18
Dependencias principales:.....	18
Credenciales para el Ingreso.....	20
Navegación de menús.....	20
Menú Principal.....	20
Bloque 2: Mecanismo de Backups y Resguardo.....	28
Introducción.....	28
Requisitos previos.....	28
Script completo.....	28
Descripción del script.....	29
Ejecución del script.....	29
RTO/RPO.....	30
Conclusión.....	31

Conexion al sistema y Base de datos

Desde mongosh:

pegar en la terminal el siguiente connection string:

```
mongosh "mongodb+srv://cluster0.dsobdqq.mongodb.net/gestionTurnos" --apiVersion 1  
--username LectorInvitado
```

Cuando solicite la Contraseña ingresar:

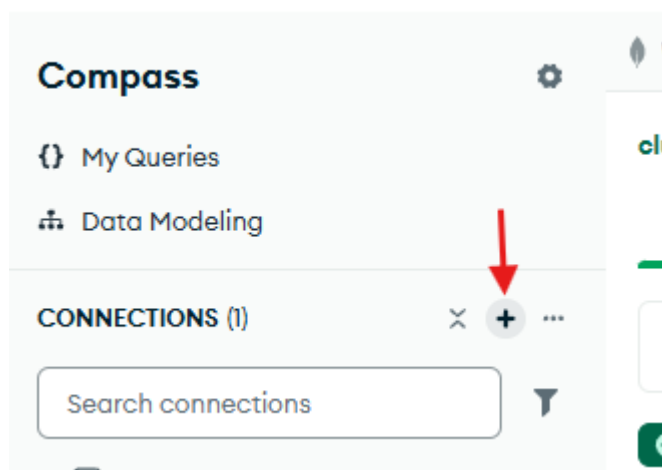
Password123

```
PS C:\Users\Matias> mongosh "mongodb+srv://cluster0.dsobdqq.mongodb.net/gestionTurnos" --apiVersion 1 --username LectorInvitado  
Enter password: *****  
Current Mongosh Log ID: 6a1790540b8395d0ed3682d0  
Connecting to:      mongodb+srv://<credentials>@cluster0.dsobdqq.mongodb.net/gestionTurnos?appName=mongosh+2.8.2  
Using MongoDB:      8.0.23 (API Version 1)  
Using Mongosh:       2.8.2  
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell  
  
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/  
  
Atlas atlas-yold74-shard-0 [primary] gestionTurnos> |
```

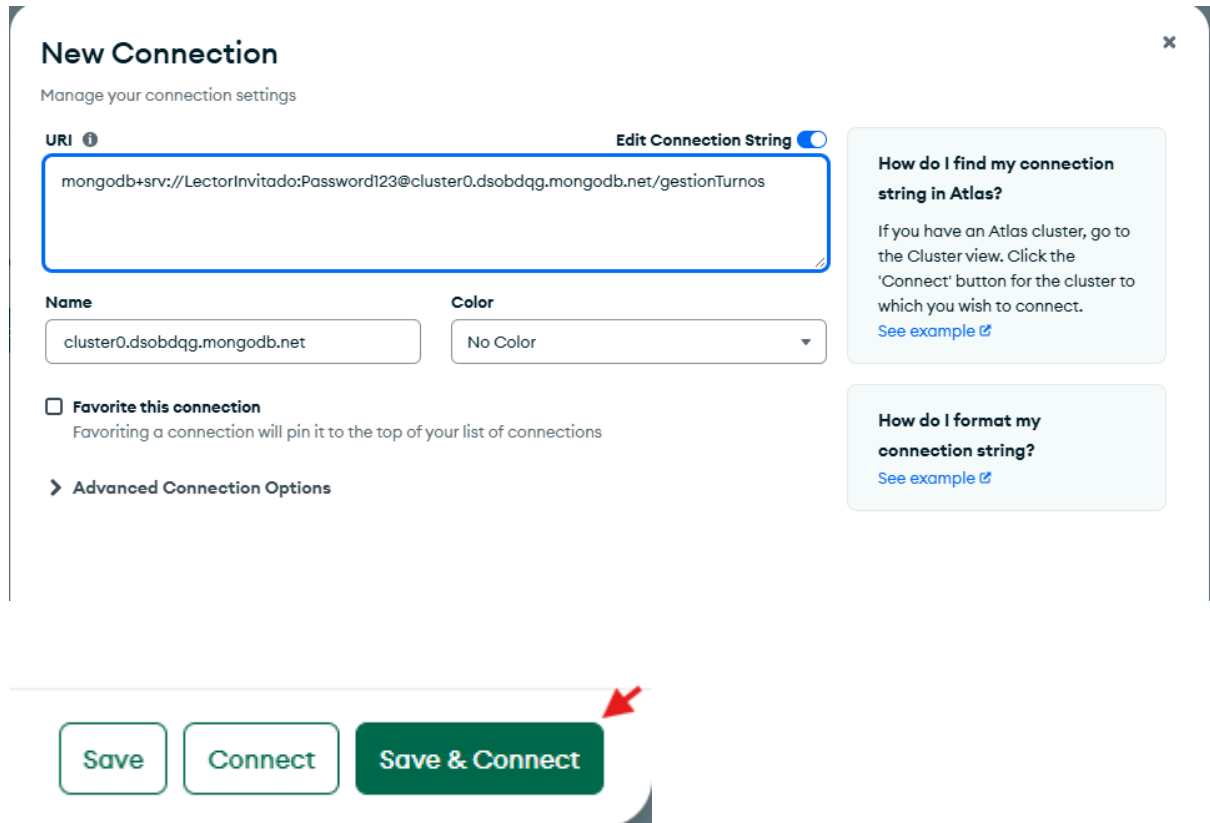
Desde MongoDB Compass:

Ir a agregar nueva conexión y pegar:

mongodb+srv://LectorInvitado:[Password123@cluster0.dsobdqq.mongodb.net/](https://www.mongodb.com/try/download/shell)



Pegar el Connection String y seleccionar Save & Connect



New Connection

Manage your connection settings

URI ⓘ Edit Connection String 

mongodb+srv://LectorInvitado:Password123@cluster0.dsobdqq.mongodb.net/gestionTurnos

Name cluster0.dsobdqq.mongodb.net Color No Color

☐ Favorite this connection
Favoriting a connection will pin it to the top of your list of connections

> Advanced Connection Options

How do I find my connection string in Atlas?
If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.
[See example](#)

How do I format my connection string?
[See example](#)

Save Connect **Save & Connect**

De esta manera queda conectada la Base de Datos en MongoDB Compass

Credenciales de Administrador para el script con el CRUD de la base de datos:

Usuario: TuturnoAdmin

Contraseña: Admin123

Introducción del sistema

Este proyecto implementa un sistema de gestión de usuarios desarrollado en Node.js, utilizando MongoDB como base de datos NoSQL y Mongoose como ODM. El sistema funciona mediante un menú interactivo en consola, permitiendo realizar operaciones CRUD completas:

- Crear usuarios
- Listar usuarios activos
- Actualizar datos
- Realizar baja lógica
- Validar datos ingresados

El objetivo es aplicar conceptos de Bases de Datos II, integrando:

- Modelado de datos
- Validaciones
- Operaciones CRUD
- Manejo de errores
- Modularización del código

Tecnologías utilizadas

Node.js - Entorno de ejecución JavaScript.

MongoDB - Base de datos NoSQL orientada a documentos.

Mongoose - ODM para modelado y consultas.

readline - Módulo de Node.js para interacción por consola.

ECMAScript Modules (ESM) - Uso de import / export.

JavaScript moderno (async/await) - Manejo claro de asincronía.

Arquitectura del proyecto

Proyecto

|

├─ MenuPrincipal.js → Menú interactivo y flujo del sistema

├─ TPI_parte2_mongoose.js → Conexión, modelo y funciones CRUD

├─ package.json → Dependencias y configuración

└─ node_modules/ → Librerías instaladas

Descripción de archivos

- **index.js**

Solicita credenciales al usuario mediante consola usando readline.

Construye dinámicamente la URI (Uniform Resource Identifier) de conexión a MongoDB Atlas.

Conecta a MongoDB usando Mongoose.

Inicializa el flujo del programa llamando a mostrarMenu() solo después de una conexión exitosa.

Maneja errores de conexión y finalizar el proceso si algo falla.

- **MenuPrincipal.js**

Maneja el menú, validaciones y flujo de interacción con el usuario.

- **TPI_parte2_mongoose.js**

Contiene:

Conexión a MongoDB

Schema y modelo Usuario

Funciones CRUD

Baja lógica

Entidades del sistema

El sistema se compone de cuatro entidades principales que representan los actores y operaciones de la gestión. Cada una cumple un rol específico dentro del modelo de datos:

- **Usuario:** Representa a la persona que solicita un turno. Contiene información básica de contacto como nombre, email y teléfono. Se gestiona con baja lógica para mantener historial de registros.
- **Profesional:** Representa al especialista que brinda servicios. Incluye nombre, especialidad y un conjunto de horarios de atención. Es clave para vincular servicios y turnos.
- **Servicio:** Define la prestación ofrecida por un profesional, con nombre, duración y precio. Se asocia directamente a un profesional y permite organizar la oferta disponible.
- **Turno:** Es la entidad que unifica todo el sistema. Relaciona a un usuario, un profesional y un servicio en una fecha y hora determinada. Además, guarda un **historial del servicio** en el momento de la creación, lo que asegura un registro histórico (precio, duración y datos del profesional en el momento creado) incluso si el servicio cambia posteriormente. De esta manera, el turno funciona como el núcleo integrador que conecta todas las entidades y garantiza la trazabilidad de la información.

Modelo de datos

Usuarios

Schema Usuarios

```
const UsuarioSchema = new mongoose.Schema({  
  nombre: { type: String, required: true },  
  email: { type: String, required: true, unique: true },  
  telefono: { type: String, required: true },  
  activo: { type: Boolean, default: true }  
}, { timestamps: true });
```


Descripción de campos

Campo	Tipo	Descripción
nombre	String	Nombre del usuario
email	String	Email único y validado
telefono	String	Teléfono de contacto
activo	Boolean	Indica si el usuario está activo (baja lógica)
createdAt	Date	Fecha de creación
updatedAt	Date	Fecha de actualización

Profesionales

Schema Profesionales

```
const horarioSchema = new mongoose.Schema({  
  dia: { type: String, required: true },  
  desde: { type: String, required: true },  
  hasta: { type: String, required: true }  
}, { _id: false });
```

```
const ProfesionalSchema = new mongoose.Schema({  
  nombre: { type: String, required: true },  
  especialidad: { type: String, required: true },  
  horarios: { type: [horarioSchema], required: true },  
  activo: { type: Boolean, default: true }  
}, { timestamps: true });
```

Descripción de campos

Campo	Tipo	Descripción
nombre	String	Nombre del profesional
especialidad	String	Especialidad médica
horarios	Array	Lista de días y franjas horarias de atención
activo	Boolean	Estado lógico (true = activo, false = baja)
createdAt	Date	Fecha de creación
updatedAt	Date	Fecha de actualización

Servicios

Schema Servicios

```
const ServicioSchema = new mongoose.Schema({  
  profesionalId: { type: mongoose.Schema.Types.ObjectId, ref: "Profesional", required: true },  
  nombre: { type: String, required: true },  
  duracion: { type: Number, required: true }, // minutos  
  precio: { type: Number, required: true },  
  activo: { type: Boolean, default: true }  
}, { timestamps: true });
```

Descripción de campos

Campo	Tipo	Descripción
profesionalId	ObjectId	Referencia al profesional que brinda el servicio
nombre	String	Nombre del servicio
duracion	Number	Duración en minutos

Campo	Tipo	Descripción
precio	Number	Precio en pesos
activo	Boolean	Estado lógico (true = activo, false = baja)
createdAt	Date	Fecha de creación
updatedAt	Date	Fecha de actualización

Turnos

Schema Turnos

```
const TurnoSchema = new mongoose.Schema({

  usuarioId: { type: mongoose.Schema.Types.ObjectId, ref: "Usuario",
required: true },

  profesionalId: { type: mongoose.Schema.Types.ObjectId, ref:
"Profesional", required: true },

  servicioId: { type: mongoose.Schema.Types.ObjectId, ref: "Servicio",
required: true },

  servicioSnapshot: {

    _id: { type: mongoose.Schema.Types.ObjectId, required: true },

    nombre: String,

    duracion: Number,

    precio: Number,

    profesional: { nombre: String, especialidad: String }

  },

  fechaInicio: { type: Date, required: true },

  estado: { type: String, enum: ["pendiente", "confirmado",
"cancelado"], default: "pendiente" },

  observaciones: String,
```

```
    activo: { type: Boolean, default: true }  
  }, { timestamps: true });
```

Descripción de campos

Campo	Tipo	Descripción
usuarioid	ObjectId	Referencia al usuario
profesionalid	ObjectId	Referencia al profesional
servicioid	ObjectId	Referencia al servicio
servicioSnapshot	Object	Copia del servicio en el momento de creación
fechaInicio	Date	Fecha y hora del turno
estado	String	Estado del turno (pendiente, confirmado, cancelado)
observaciones	String	Notas adicionales
activo	Boolean	Estado lógico
createdAt	Date	Fecha de creación
updatedAt	Date	Fecha de actualización

Funciones CRUD

En el sistema se implementan operaciones CRUD (Create, Read, Update, Delete) para cada entidad del modelo de datos: **Usuarios, Profesionales, Servicios y Turnos**. Estas funciones permiten gestionar la información de manera estructurada y segura, aplicando **baja lógica** en lugar de eliminación física para preservar la integridad de los registros.

Cada operación está desarrollada con **Node.js y Mongoose**, utilizando **async/await** para un manejo claro de la asincronía y validaciones específicas en cada caso. A continuación se detallan las funciones CRUD de cada entidad, acompañadas de su código y explicación.

Usuarios

Crear usuario (CREATE)

```
export async function crearUsuario(data) {  
    return await Usuario.create(data);  
}
```

Listar usuarios activos (READ)

```
export async function listarUsuarios() {  
    return await Usuario.find({ activo: true });  
}
```

Actualizar un usuario (UPDATE)

```
export async function actualizarUsuario(id, campos) {  
    return await Usuario.findByIdAndUpdate(id, campos, { returnDocument:  
    "after" });  
}
```

Dar de baja un usuario (DELETE) Baja logica

```
export async function bajaUsuario(id) {  
    return await Usuario.findOneAndUpdate(  
        { _id: id },  
        { $set: { activo: false } },  
        { returnDocument: "after" }  
    );  
}
```

Profesionales

Crear profesional (CREATE)

```
export async function crearProfesional(data) {  
  
    return await Profesional.create(data);  
  
}
```

Listar profesionales activos (READ)

```
export async function listarProfesionales() {  
  
    return await Profesional.find({ activo: true });  
  
}
```

Actualizar un profesional (UPDATE)

```
export async function actualizarProfesional(id, campos) {  
  
    return await Profesional.findByIdAndUpdate(id, campos, {  
returnDocument: "after" });  
  
}
```

Dar de baja un profesional (DELETE) Baja lógica

```
export async function bajaProfesional(id) {  
  
    return await Profesional.findByIdAndUpdate(id, { activo: false }, {  
returnDocument: "after" });  
  
}
```

Servicios

Crear servicio (CREATE)

```
export async function crearServicio(data) {  
  
    return await Servicio.create(data);  
  
}
```

Listar servicios activos (READ)

```
export async function listarServicios() {  
  
    return await Servicio.find({ activo: true  
}).populate("profesionalId", "nombre especialidad");  
  
}
```

Listar servicios por profesional (READ)

```
export async function listarServiciosPorProfesional(profesionalId) {  
  
    return await Servicio.find({ profesionalId, activo: true });  
  
}
```

Actualizar un servicio (UPDATE)

```
export async function actualizarServicio(id, campos) {  
  
    return await Servicio.findByIdAndUpdate(  
  
        id,  
  
        campos,  
  
        { returnDocument: "after" }  
  
    );  
  
}
```

Dar de baja un servicio (DELETE) Baja lógica

```
export async function bajaServicio(id) {  
  
    return await Servicio.findByIdAndUpdate(  
  
        id,  
  
        { activo: false },  
  
        { returnDocument: "after" }  
  
    );  
  
}
```

Turnos

Crear turno (CREATE)

```
export async function crearTurno({ usuarioId, profesionalId, servicioId,
fechaInicio, observaciones, estado }) {

    const servicio = await
Servicio.findById(servicioId).populate("profesionalId");

    if (!servicio) throw new Error("Servicio no encontrado");

    // histórico del servicio

    const snapshot = {

        _id: servicio._id,

        nombre: servicio.nombre,

        duracion: servicio.duracion,

        precio: servicio.precio,

        profesional: {

            nombre: servicio.profesionalId.nombre,

            especialidad: servicio.profesionalId.especialidad,

        }

    };

    return await Turno.create({

        usuarioId,

        profesionalId,

        servicioId,

        servicioSnapshot: snapshot,

        fechaInicio,

        observaciones,
```



```
        estado,  
        activo: true  
    });  
}
```

Listar turnos activos (READ)

```
export async function listarTurnos() {  
    return await Turno.find({ activo: true })  
        .populate("usuarioId", "nombre email")  
        .populate("profesionalId", "nombre especialidad");  
}
```

Actualizar un turno (UPDATE)

```
export async function actualizarTurno(id, campos) {  
    return await Turno.findByIdAndUpdate(  
        id,  
        campos,  
        { returnDocument: "after" }  
    );  
}
```

Dar de baja un turno (DELETE) Baja lógica

```
export async function bajaTurno(id) {  
    return await Turno.findByIdAndUpdate(  
        id,  
        { activo: false },
```

```
    { returnDocument: "after" }  
  
  );  
  
}
```

Guía de Uso del Sistema

Requisitos previos

Para ejecutar el sistema es necesario tener:

- **Node.js** (versión 18 o superior recomendada).
 - **MongoDB Atlas** (El sistema ya conecta a su base de datos propia)
 - **Mongosh** Opcional para poder visualizar los cambios de la Base de Datos sin utilizar Atlas.
 - **pnpm** como gestor de paquetes (más seguro y eficiente que npm).
 - Conexión a internet para acceder a la base de datos en la nube.
-

Instalación de dependencias

1. **Instalar pnpm** (si no lo tenés):

```
npm install -g pnpm
```

2. **Inicializar el proyecto** (si aún no existe package.json):

```
pnpm init
```

4. **Instalar Mongoose**

```
pnpm add mongoose
```

5. **Instalar todas las dependencias necesarias:**

```
pnpm add mongoose readline
```

Dependencias principales:

- **mongoose** ODM para modelado de datos en MongoDB.
- **readline** Módulo para interacción por consola.

6. Ejecución del sistema

Iniciar el programa desde la raíz del proyecto:

```
node index.js
```

También es posible utilizar **pnpm start** si se configura el package.json como muestra el siguiente paso.

Nota sobre ECMAScript Modules (ESM)

(El warning que aparece al ejecutar el script)

```
(node:4896) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of
file:///RUTA/index.js is not specified and it doesn't parse as CommonJS.
```

Reparsing as ES module because module syntax was detected. This incurs a performance overhead.

To eliminate this warning, add "type": "module" to RUTA\package.json

El proyecto utiliza sintaxis moderna de JavaScript (import / export), lo que requiere que Node.js interprete los archivos como ES Modules.

Para evitar advertencias al ejecutar index.js, es necesario agregar la propiedad **"type": "module"** en el archivo package.json.

Ejemplo:

```
{
  "name": "parte-1-script",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "mongoose": "^9.7.3",
```

```
    "readline": "^1.3.0"

  }

}
```

Con esta configuración, Node.js reconoce automáticamente la sintaxis ESM y el warning desaparece. Además, se habilita el comando **pnpm start** (declarando el **"main": "index.js"**) para ejecutar el sistema de forma más sencilla.

Credenciales para el Ingreso

Al ejecutar index.js

Se solicitarán las credenciales de conexión a MongoDB Atlas (usuario y contraseña).

Ingresar

Usuario: TuturnoAdmin

Contraseña: Admin123

Se construirá dinámicamente la URI de conexión en el index.

En caso de error, se muestra un mensaje y se finaliza el proceso.

Una vez conectado, se mostrará el Menú Principal.

Navegación de menús

El sistema funciona mediante un menú interactivo en consola:

Menú Principal

```
=== MENÚ PRINCIPAL ===

1. Gestión de Usuarios
2. Gestión de Profesionales
3. Gestión de Servicios
4. Gestión de Turnos
```

5. Salir

Cada opción abre un submenú con operaciones CRUD:

Usuarios - Crear, listar, actualizar, dar de baja.

Profesionales - Crear, listar, actualizar, dar de baja.

Servicios - Crear, listar, actualizar, dar de baja.

Turnos - Crear, listar, actualizar, dar de baja.

La navegación se realiza ingresando el número de la opción deseada. En cada submenú se pueden volver al menú principal o continuar con nuevas operaciones.

Cuando se solicita una ID, copiar de la tabla correspondiente y pegarla.

Capturas:

Creacion de usuario:

```
=== MENÚ DE USUARIOS ===
1. Crear usuario
2. Listar usuarios
3. Actualizar usuario
4. Dar de baja usuario
5. Salir
.....

Elegí una opción: 1

.....

Nombre: Esteban Gutierrez

Email: esteban.gutierrez@mail.com

Teléfono: 112356485
Usuario creado: {
  nombre: 'Esteban Gutierrez',
  email: 'esteban.gutierrez@mail.com',
  telefono: '112356485',
  activo: true,
  _id: new ObjectId('6a3ea4461fe410a767055970'),
  createdAt: 2026-06-26T16:09:42.260Z,
  updatedAt: 2026-06-26T16:09:42.260Z,
  __v: 0
}
```

```
Usuario creado: {
  nombre: 'Esteban Gutierrez',
  email: 'esteban.gutierrez@mail.com',
  telefono: '112356485',
  activo: true,
  _id: new ObjectId('6a3ea4461fe410a767055970'),
  createdAt: 2026-06-26T16:09:42.260Z,
  updatedAt: 2026-06-26T16:09:42.260Z,
  __v: 0
}
```

Listar usuarios:

Elegí una opción: 2

.....

Usuarios activos:

(index)	id	nombre	email	telefono
0	'6a0f9177344626dc767c0b48'	'Hugo Catalán'	'hugo@mail.com'	'3421234567'
1	'6a0f91ea344626dc767c0b4a'	'Matias Carro'	'matias@mail.com'	'3419876543'
2	'6a17784aff7c613bad3682d3'	'Nicolas Diaz'	'nicolas.diaz@mail.com'	'3491122334'
3	'6a375b4f0fa21ff331d8fc07'	'Hugo Catalán TPI Parte 2'	'hugo_tpi-parte2@mail.com'	'3421234567'
4	'6a375b4f0fa21ff331d8fc08'	'Matias Carro TPI Parte 2'	'matias_tpi-parte2@mail.com'	'3419876543'
5	'6a3ea0a1ce638352b45071b7'	'Juan Perez'	'juan.perez@mail.com'	'112356485'
6	'6a3ea4461fe410a767055970'	'Esteban Gutierrez'	'esteban.gutierrez@mail.com'	'112356485'

Verificacion de input

Elegí una opción: 4

.....

(index)	id	nombre	email	telefono
0	'6a0f9177344626dc767c0b48'	'Hugo Catalán'	'hugo@mail.com'	'3421234567'
1	'6a0f91ea344626dc767c0b4a'	'Matias Carro'	'matias@mail.com'	'3419876543'
2	'6a17784aff7c613bad3682d3'	'Nicolas Diaz'	'nicolas.diaz@mail.com'	'3491122334'
3	'6a375b4f0fa21ff331d8fc07'	'Hugo Catalán TPI Parte 2'	'hugo_tpi-parte2@mail.com'	'3421234567'
4	'6a375b4f0fa21ff331d8fc08'	'Matias Carro TPI Parte 2'	'matias_tpi-parte2@mail.com'	'3419876543'
5	'6a3ea0a1ce638352b45071b7'	'Juan Perez'	'juan.perez@mail.com'	'112356485'
6	'6a3ea4461fe410a767055970'	'Esteban Gutierrez'	'esteban.gutierrez@mail.com'	'112356485'

ID del usuario a dar de baja: 5

El ID ingresado no es válido. Debe ser un ObjectId de 24 caracteres.

.....

Baja de usuario

(index)	id	nombre	email	telefono
0	'6a0f9177344626dc767c0b48'	'Hugo Catalán'	'hugo@mail.com'	'3421234567'
1	'6a0f91ea344626dc767c0b4a'	'Matias Carro'	'matias@mail.com'	'3419876543'
2	'6a17784aff7c613bad3682d3'	'Nicolas Díaz'	'nicolas.diaz@mail.com'	'3491122334'
3	'6a375b4f0fa21ff331d8fc07'	'Hugo Catalán TPI Parte 2'	'hugo_tpi-parte2@mail.com'	'3421234567'
4	'6a375b4f0fa21ff331d8fc08'	'Matias Carro TPI Parte 2'	'matias_tpi-parte2@mail.com'	'3419876543'
5	'6a3ea0a1ce638352b45071b7'	'Juan Perez'	'juan.perez@mail.com'	'112356485'
6	'6a3ea4461fe410a767055970'	'Esteban Gutierrez'	'esteban.gutierrez@mail.com'	'112356485'

ID del usuario a dar de baja: 6a17784aff7c613bad3682d3
(node:32368) [MONGODB] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated: 'after' instead.

Usuario dado de baja: {
 _id: new ObjectId('6a17784aff7c613bad3682d3'),
 nombre: 'Nicolas Diaz',
 email: 'nicolas.diaz@mail.com',
 telefono: '3491122334',
 activo: false,
 createdAt: 2026-05-27T20:02:00.000Z,
 updatedAt: 2026-06-26T16:34:21.896Z
}

ID del usuario a dar de baja: 6a17784aff7c613bad3682d3

Usuario dado de baja: {
 _id: new ObjectId('6a17784aff7c613bad3682d3'),
 nombre: 'Nicolas Diaz',
 email: 'nicolas.diaz@mail.com',
 telefono: '3491122334',
 activo: false,
 createdAt: 2026-05-27T20:02:00.000Z,
 updatedAt: 2026-06-26T16:34:21.896Z
}

Creacion de profesionales

```
Profesional creado: {
  nombre: 'Dr. German Ruiz',
  especialidad: 'Kinesiologo',
  horarios: [ { dia: 'Lunes', desde: '12:00', hasta: '18:00' } ],
  activo: true,
  _id: new ObjectId('6a40596034cb79450a572c24'),
  createdAt: 2026-06-27T23:14:40.392Z,
  updatedAt: 2026-06-27T23:14:40.392Z,
  __v: 0
}
```

=== MENÚ DE PROFESIONALES ===

1. Crear profesional
2. Listar profesionales
3. Actualizar profesional
4. Dar de baja profesional
5. Salir

.....

Elegí una opción: 1

.....

Ingrese los datos del nuevo profesional:

Nombre: Dr. German Ruiz

Especialidad: Kinesiologo

Día de atención: Lunes

Desde (HH:mm): 12:00

Hasta (HH:mm): 18:00

```
Profesional creado: {
  nombre: 'Dr. German Ruiz',
  especialidad: 'Kinesiologo',
  horarios: [ { dia: 'Lunes', desde: '12:00', hasta: '18:00' } ],
  activo: true,
  _id: new ObjectId('6a40596034cb79450a572c24'),
  createdAt: 2026-06-27T23:14:40.392Z,
  updatedAt: 2026-06-27T23:14:40.392Z,
  __v: 0
}
```

Turno creado

```
Turno creado:
{
  usuarioId: new ObjectId('6a0f91ea344626dc767c0b4a'),
  profesionalId: new ObjectId('6a0f927f344626dc767c0b51'),
  servicioId: new ObjectId('6a0f9368344626dc767c0b5a'),
  servicioSnapshot: {
    _id: new ObjectId('6a0f9368344626dc767c0b5a'),
    nombre: 'Sesión de kinesiología',
    duracion: 60,
    precio: 20000,
    profesional: { nombre: 'Carlos Martínez', especialidad:
'Kinesiología' }
  },
  fechaInicio: 2026-08-05T19:45:00.000Z,
  estado: 'confirmado',
  observaciones: 'Turno 1 de 10',
  activo: true,
  _id: new ObjectId('6a4087d37d8141fca94dadba'),
  createdAt: 2026-06-28T02:32:51.408Z,
  updatedAt: 2026-06-28T02:32:51.408Z,
  __v: 0
}
```

Ingrese ID del servicio: 6a0f9368344626dc767c0b5a

Ingrese fecha (YYYY-MM-DD, ejemplo: 2026-07-01): 2026-08-05

Ingrese hora (HH:mm, ejemplo: 14:30): 16:45

Observaciones: Turno 1 de 10

Estado (1. pendiente, 2. confirmado, 3. cancelado): 2

```
Turno creado:
{
  usuarioId: new ObjectId('6a0f91ea344626dc767c0b4a'),
  profesionalId: new ObjectId('6a0f927f344626dc767c0b51'),
  servicioId: new ObjectId('6a0f9368344626dc767c0b5a'),
  servicioSnapshot: {
    _id: new ObjectId('6a0f9368344626dc767c0b5a'),
    nombre: 'Sesión de kinesiología',
    duracion: 60,
    precio: 20000,
    profesional: { nombre: 'Carlos Martínez', especialidad: 'Kinesiología' }
  },
  fechaInicio: 2026-08-05T19:45:00.000Z,
  estado: 'confirmado',
  observaciones: 'Turno 1 de 10',
  activo: true,
  _id: new ObjectId('6a4087d37d8141fca94dadba'),
  createdAt: 2026-06-28T02:32:51.408Z,
  updatedAt: 2026-06-28T02:32:51.408Z,
  __v: 0
}
```

Turno cargado en MongoDB Atlas

```
▶ {
  _id: ObjectId('6a4087d37d8141fca94dadba')
  usuarioId: ObjectId('6a0f91ea344626dc767c0b4a')
  profesionalId: ObjectId('6a0f927f344626dc767c0b51')
  servicioId: ObjectId('6a0f9368344626dc767c0b5a')
  servicioSnapshot: Object
    _id: ObjectId('6a0f9368344626dc767c0b5a')
    nombre: "Sesión de kinesiología"
    duracion: 60
    precio: 20000
    profesional: Object
      nombre: "Carlos Martínez"
      especialidad: "Kinesiología"
  fechaInicio: 2026-08-05T19:45:00.000+00:00
  estado: "confirmado"
  observaciones: "Turno 1 de 10"
  activo: true
  createdAt: 2026-06-28T02:32:51.408+00:00
  updatedAt: 2026-06-28T02:32:51.408+00:00
  __v: 0
}
```

Bloque 2: Mecanismo de Backups y Resguardo

Introducción

Este script simula una tarea de administración de servidores, automatizando el respaldo físico de datos mediante la línea de comandos del sistema operativo.

El objetivo es garantizar la disponibilidad y recuperación de la información en caso de fallos, aplicando buenas prácticas de **RTO (Recovery Time Objective)** y **RPO (Recovery Point Objective)**.

Requisitos previos

- **Sistema operativo Windows** con PowerShell.
 - **MongoDB Database Tools** instaladas (incluyendo mongodump.exe).
 - **Acceso a MongoDB Atlas** con credenciales válidas.
 - Conexión a internet para acceder al clúster remoto.
-

Script completo

```
# Crear carpeta principal de resguardos (si no existe)
$basePath = ".\resguardos_tpi"
if (!(Test-Path $basePath)) {
    New-Item -ItemType Directory -Path $basePath | Out-Null
}

# Crear subcarpeta con fecha actual (YYYY-MM-DD)
$dateFolder = (Get-Date -Format "yyyy-MM-dd")
$backupPath = Join-Path $basePath $dateFolder
New-Item -ItemType Directory -Path $backupPath -Force | Out-Null

# Ejecutar mongodump con conexión a Atlas
& "C:\Program Files\MongoDB\Tools\100\bin\mongodump.exe"
--uri="mongodb+srv://TuturnoAdmin:Admin123@cluster0.dsobdqg.mongodb.net/
gestionTurnos" --out $backupPath

Write-Host "Respaldo completado en $backupPath"
```

Descripción del script

El script **backup.ps1** realiza las siguientes operaciones:

1. **Creación de carpeta principal de resguardo**

- Se genera la carpeta **resguardos_tpi** en el directorio actual si no existe.

2. **Creación de subcarpeta con fecha actual (YYYY-MM-DD)**

- Se organiza cada respaldo en una carpeta con la fecha correspondiente.

3. **Ejecución de mongodump**

- Se conecta al clúster de Atlas usando las credenciales:

- **Usuario:** TuturnoAdmin

- **Contraseña:** Admin123

Nota importante Para esta version del script debido a que es algo educativo se almacena el string completo con el admin, esto no es algo que se debe utilizar en un entorno de produccion real y debe ser eliminado del script.

Una opcion es guardar las credenciales como variables de entorno en un archivo que nunca se suba a ningun repositorio o servicio web.

- Descarga la base de datos **gestionTurnos** dentro de la carpeta creada.

4. **Confirmación en consola**

- Muestra el mensaje:

```
Respaldo completado en .\resguardos_tpi\YYYY-MM-DD
```

Ejecución del script

1. Guardar el archivo como **backup.ps1**.
2. Abrir PowerShell en la carpeta del proyecto.
3. Ejecutar:

```
.\backup.ps1
```

Script en funcionamiento:

```
PS E:\UTN TPU\2DO AÑO\Tercer Cuatrimestre\Bases de Datos 2\TPI parte 2> .\backup_TPI_p2.ps1
2026-06-21T00:43:05.704-0300 writing 'gestionTurnos.profesionales' to 'resguardos_tpi_parte2\2026-06-21\gestionTurnos\profesionales.bson'
2026-06-21T00:43:05.715-0300 writing 'gestionTurnos.servicios' to 'resguardos_tpi_parte2\2026-06-21\gestionTurnos\servicios.bson'
2026-06-21T00:43:05.715-0300 writing 'gestionTurnos.usuarios' to 'resguardos_tpi_parte2\2026-06-21\gestionTurnos\usuarios.bson'
2026-06-21T00:43:05.715-0300 writing 'gestionTurnos.turnos' to 'resguardos_tpi_parte2\2026-06-21\gestionTurnos\turnos.bson'
2026-06-21T00:43:05.912-0300 done dumping 'gestionTurnos.profesionales' (7 documents)
2026-06-21T00:43:05.955-0300 done dumping 'gestionTurnos.turnos' (3 documents)
2026-06-21T00:43:05.988-0300 done dumping 'gestionTurnos.servicios' (6 documents)
2026-06-21T00:43:05.992-0300 done dumping 'gestionTurnos.usuarios' (6 documents)
Respaldo completado en .\resguardos_tpi_parte2\2026-06-21
PS E:\UTN TPU\2DO AÑO\Tercer Cuatrimestre\Bases de Datos 2\TPI parte 2> |
```

Salida de la consola:

```
2026-06-26T14:20:59.067-0300 writing 'gestionTurnos.usuarios' to
'resguardos_tpi\2026-06-26\gestionTurnos\usuarios.bson'
2026-06-26T14:20:59.081-0300 writing 'gestionTurnos.profesionales' to
'resguardos_tpi\2026-06-26\gestionTurnos\profesionales.bson'
2026-06-26T14:20:59.081-0300 writing 'gestionTurnos.servicios' to
'resguardos_tpi\2026-06-26\gestionTurnos\servicios.bson'
2026-06-26T14:20:59.081-0300 writing 'gestionTurnos.turnos' to
'resguardos_tpi\2026-06-26\gestionTurnos\turnos.bson'
2026-06-26T14:21:00.093-0300 done dumping 'gestionTurnos.usuarios' (8
documents)
2026-06-26T14:21:00.195-0300 done dumping
'gestionTurnos.profesionales' (7 documents)
2026-06-26T14:21:00.278-0300 done dumping 'gestionTurnos.servicios'
(6 documents)
2026-06-26T14:21:00.368-0300 done dumping 'gestionTurnos.turnos' (3
documents)
Respaldo completado en .\resguardos_tpi\2026-06-26
PS C:\Matias\UTN\Tercer Cuatrimestre\Bases de Datos 2\TPI parte
2\Archivos TPI parte 2\Parte 2 - Backups>
```

RTO/RPO

En el proyecto de gestión de turnos, el análisis de RTO y RPO se traduce en la capacidad de garantizar continuidad y recuperación de datos frente a fallos. El RTO (Recovery Time Objective) es bajo, ya que los respaldos generados con **mongodump** permiten restaurar la base de datos rápidamente mediante **mongorestore**, asegurando que el sistema pueda volver a estar operativo en relativamente poco tiempo.

RPO (Recovery Point Objective) depende de la frecuencia de ejecución del script de backup: si se realiza diariamente, el máximo de pérdida de información sería de 24 horas de turnos o registros. Sin embargo, al automatizar la tarea con un programador de tareas, este valor puede reducirse considerablemente.

Al tratarse de un sistema de turnos (y en el ejemplo utilizado específicamente turnos médicos) el RPO debe ser frecuente, preferentemente de una hora, para reducir lo mas posible la pérdida de datos. Como mongodump realiza copias enteras de la base de datos, estos backups graduales podrían ser manejados con otra herramienta que si lo permita, como MongoDB Atlas Backups. Esto permitiría tener backups cada hora para recuperación, pero además se podría mantener el mongodump para un resguardo completo cada 24 hs.

Conclusión

La implementación de la comunicación Cliente-Servidor en este proyecto permitió comprender de manera práctica cómo interactúan las aplicaciones con una base de datos remota.

Uno de los principales desafíos fue garantizar la consistencia de los datos y manejar correctamente la asincronía en Node.js mediante async/await. También resultó clave aplicar validaciones en cada operación para evitar errores comunes como IDs inválidos, emails repetidos o formatos incorrectos.

Otro aspecto complejo fue la modularización del sistema, separando la lógica de conexión, los modelos y los menús interactivos. Esto facilitó la escalabilidad y el mantenimiento del código.

La entidad Turno es el núcleo del sistema, ya que unifica usuarios, profesionales y servicios en una sola operación. El uso del histórico del servicio en el momento de la creación fue un aprendizaje importante: asegura la trazabilidad histórica y evita inconsistencias si los datos del servicio cambian posteriormente.

Además, se incorporó un mecanismo de backup automatizado mediante **mongodump**, que permite resguardar la base de datos en carpetas organizadas por fecha, garantizando la recuperación ante fallos. Permitiendo un RTO rápido debido a la restauración de mongorestore.